

App Dev 2 Project Report

1. Student Details

- **Name:** Vaibhav Satish
- **Roll Number:** 21F3002068
- **Email:** 21f3002068@ds.study.iitm.ac.in
- **About Me:** I completed my BSc in Statistics from Aligarh Muslim University. I am currently a standalone student at the IIT Madras BS Degree program. With an interest in web application development and data-driven technologies, I am passionate about Responsible AI Research and plan to do research and publish a paper in this field by next year.

2. Project Details

- **Project Title:** ParkEase - A Smart Parking Management System
- **Problem Statement:** To design and build a comprehensive web application to streamline the management of parking facilities. The system aims to solve key problems for both administrators and end-users by providing a centralized platform for managing parking lots, users, and reservations. It addresses challenges such as inefficient space utilization, a lack of real-time data for administrators, and difficulties for drivers in finding available parking.
- **Approach:** The project was developed with a modern technology stack, featuring a **Vue.js** single-page application (SPA) for the frontend and a **Flask**-powered REST API for the backend. This architecture creates a decoupled and scalable system. The core of the project is a feature-rich admin dashboard that provides full control over the parking ecosystem. It includes:
 - CRUD (Create, Read, Update, Delete) operations for parking lots and users.
 - Real-time visualization of parking lot occupancy.
 - An advanced analytics section with interactive charts (using Chart.js) to display usage trends, revenue, and other key metrics.
 - Management of background tasks.
 - Detailed views for user and reservation data.

3. AI/LLM Declaration

I used ChatGPT 5 to help write SQLAlchemy model definitions, generate feature ideas, and also used Gemini Code Assist to create API documentation samples, yaml file, and improve variable naming consistency. The extent of AI/LLM usage is around 20–35%, limited to code suggestions and documentation formatting. All final implementation logic, debugging, integration, and enhancements were done manually.

4. Technologies and Frameworks Used

Technology / Library	Purpose
Vue.js	Core frontend framework for building a reactive and component-based Single-Page Application (SPA).

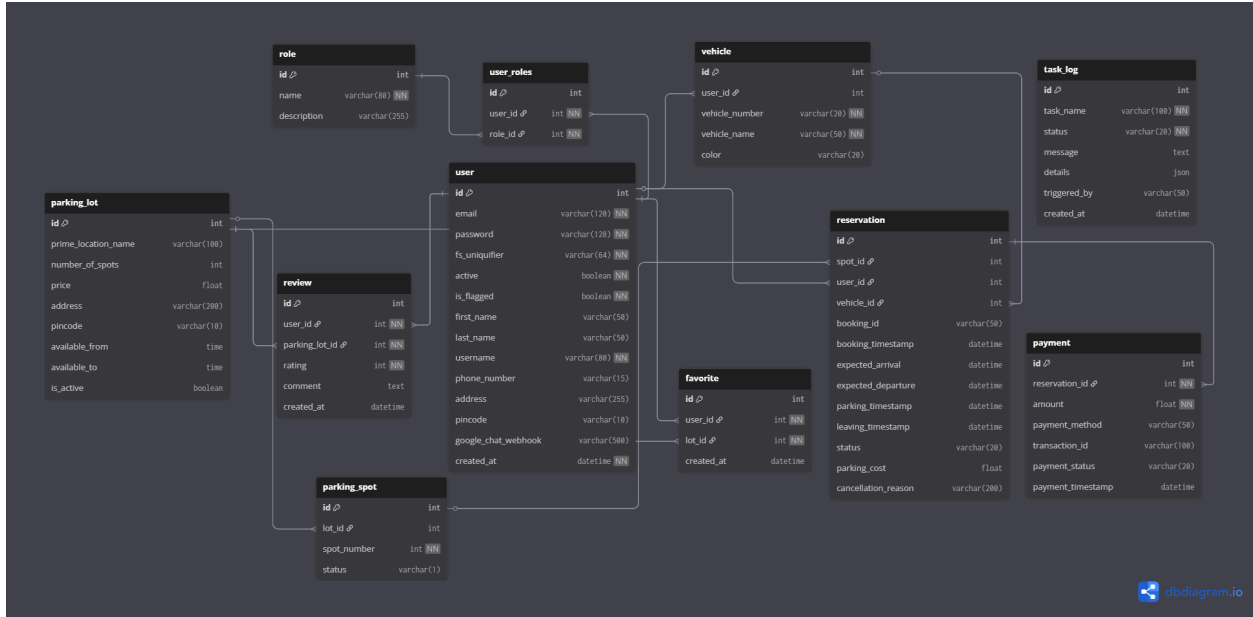
Flask	A lightweight and powerful backend web framework used to build the RESTful API.
SQLAlchemy	Object-Relational Mapper (ORM) for defining database models and interacting with the database.
Chart.js	A JavaScript library used to create dynamic and interactive charts for the analytics dashboard.
Vue Router	Handles client-side routing within the Vue.js application, enabling navigation between different views.
Token Auth (JWT)	Used for securing API endpoints and managing user sessions via authentication tokens.
Celery	(Inferred) Asynchronous task queue for handling background jobs like sending emails and running reports.
SQLite	A lightweight, serverless database used for storing application data during development.

5. Database Schema

- **Tables:**
 - **User:** Stores user profile information, credentials, roles, and status (id, username, email, password_hash, first_name, last_name, active, is_flagged).
 - **Role:** Stores user roles (e.g., 'admin', 'user').
 - **UserRoles:** A mapping table to link users to their roles (many-to-many).
 - **ParkingLot:** Contains details about each parking facility (id, name, address, pincode, capacity, price, is_active).
 - **ParkingSpot:** Represents individual spots within a parking lot (id, lot_id, spot_number, is_occupied).
 - **Reservation:** Logs all booking details, linking users, lots, and spots (id, user_id, lot_id, spot_id, vehicle_id, booking_timestamp, status, parking_cost).
 - **Vehicle:** Stores details of vehicles registered by users (id, user_id, license_plate, model, color).
 - **TaskLog:** Records the execution and outcome of background administrative tasks (id, task_name, status, message, timestamp).
- **Relationships:**
 - **One-to-Many:** User → Reservation (A user can have many reservations).
 - **One-to-Many:** User → Vehicle (A user can own multiple vehicles).

- **One-to-Many:** ParkingLot → ParkingSpot (A lot has many spots).
- **One-to-Many:** ParkingLot → Reservation (A lot can have many reservations).
- **Many-to-Many:** User ↔ Role (A user can have multiple roles, and a role can be assigned to many users).

ER Diagram



6. API Resource Endpoints

Based on the frontend code, the following API endpoints are used:

Endpoint	Method	Description
/api/admin/parking_lots	GET	Fetch all parking lots for the admin dashboard.
/api/admin/parking_lots	POST	Create a new parking lot.
/api/admin/parking_lots/<id>	PUT	Update an existing parking lot's details.
/api/admin/parking_lots/<id>	DELETE	Delete a parking lot.
/api/admin/parking_lots/<id>/toggle	POST	Activate or deactivate a parking lot.
/api/admin/users	GET	Fetch a list of all users.
/api/admin/users/export	POST	Trigger a background task to export user data as a CSV file.

/api/admin/users/<id>/details	GET	Fetch detailed information for a specific user, including vehicles and reservations.
/api/admin/users/<id>/flag	POST/DELETE	Flag or unflag a user account.
/api/admin/reservations	GET	Fetch paginated and filtered reservation data.
/api/admin/reservations/export	POST	Trigger an export of reservation data to a CSV file.
/api/admin/analytics/dashboard	GET	Retrieve aggregated data for all analytics charts and overview cards.
/api/admin/tasks/trigger/<task_name>	POST	Manually trigger a scheduled background task (e.g., cleanup, health check).

- **YAML API Definition File:** Included separately in the submission ZIP as api.yaml.

7. Architecture and Features

- **Architecture Overview:**
 - frontend/ - Contains the Vue.js source code.
 - src/views/admin/AdminDashboard.vue – The main component for the entire admin interface.
 - src/components/ – Reusable Vue components like ParkingLotVisualizer.vue, Modal.vue, and ReservationDetailModal.vue.
 - src/assets/ – Static assets, including CSS stylesheets and images.
 - src/router/ – Vue Router configuration for client-side navigation.
 - backend/ - Contains the Flask application.
 - app.py – Main Flask application entry point, configuration, and extensions.
 - models/ – SQLAlchemy database models.
 - routes/ or blueprints/ – Flask Blueprints to modularize API routes (e.g., admin_routes.py, auth_routes.py).
 - tasks/ – Definitions for background tasks managed by Celery.
- **Implemented Features:**
 - **Secure Admin Panel:** Role-based access control for the admin dashboard.
 - **Parking Lot Management:** Full CRUD functionality for parking lots, including setting capacity, pricing, and operating hours.
 - **User Management:** View a list of all users, inspect their details (vehicles, reservations), and manage their status (active/inactive, flagged).
 - **Reservation Oversight:** A comprehensive table to view, filter, and paginate through all system reservations.
 - **Data-Driven Dashboard:** An overview section with key performance indicators (KPIs) like total users, net earnings, and current occupancy rate.
 - **Advanced Analytics:** A dedicated "Summaries" tab with multiple charts visualizing:

- Revenue trends over time.
- Utilization rates per parking lot.
- Daily booking activity.
- Peak usage hours.
- **Live Visualization:** A "Bird's Eye View" of parking lots showing real-time spot occupancy.
- **Task Management:** An interface to monitor and manually trigger background administrative tasks, such as data cleanup and report generation.
- **Data Export:** Functionality to export user and reservation data to CSV, handled asynchronously to avoid blocking the UI.

8. Video Presentation

Drive Link:  ParkEase_Project_Presentation_Mad2.mp4

(Accessible to all with "View" permission)

YT Video Link: <https://youtu.be/2-0TXPrJNqE>